

WirSchiffenDas

Engine Quality Analysis

arc42 Architekturbeschreibung

Service-basierte Komponenten-Architekturen (SEKA)

Theodoros Bampis & Yunus Emre Sirin

Sommersemester 2026

Dokumenttyp: Architekturdokumentation nach arc42
Architekturstil: Microservices
Schwerpunkt: Docker und Docker Compose
Stand: 20. September 2026

Inhaltsverzeichnis

1	Einführung und Ziele	3
1.1	Aufgabenstellung	3
1.2	Architekturziele	3
1.3	Wesentliche fachliche Anforderungen	3
2	Randbedingungen	4
2.1	Technische Randbedingungen	4
2.2	Projektgrenzen	4
3	Kontextabgrenzung	4
3.1	Fachlicher Kontext	4
4	Lösungsstrategie	5
4.1	Strategic Domain-Driven Design	5
4.2	Datenhoheit und Integrationsprinzip	5
5	Bausteinsicht	6
5.1	Ebene 1: Systemübersicht	6
5.2	Externe Schnittstellen	6
5.3	Interne Schnittstellen	6
6	Laufzeitsicht	7
6.1	Happy Path	7
6.2	Fehlerfall, Circuit Breaker und Retry	7
7	Verteilungssicht	8
7.1	Deployment-Prinzipien	8
8	Querschnittliche Konzepte	8
8.1	Status- und Ergebnismodell	8
8.2	Resilience	8
8.3	Persistenz und Datenhoheit	8
9	Architekturentscheidungen	9
9.1	ADR-01: Fachlicher Schnitt statt technischer Layer	9
9.2	ADR-02: REST/HTTP für Version 1	9
9.3	ADR-03: Choreographie statt zentraler Orchestration	9
9.4	ADR-04: Circuit Breaker mit Resilience4j	9
9.5	ADR-05: Container pro Microservice	9
9.6	ADR-06: Keine Shared Persistence	9
10	Qualitätsanforderungen und Architecture Smells	9
10.1	Priorisierte Probleme und Gegenmaßnahmen	9
10.2	Qualitätsszenarien	10
11	Risiken und technische Schulden	10
12	Quellenbasis	10

1 Einführung und Ziele

1.1 Aufgabenstellung

Ausgangspunkt ist die in der Fallstudie beschriebene Analyse-Komponente von *Manufacturing Products*. Mehrere fachlich eigenständige Validierungs- und Simulationsalgorithmen sind dort eng gekoppelt. Lange Laufzeiten einzelner Analysen blockieren dadurch den gesamten Konstruktionsablauf und verschlechtern die Responsiveness der Anwendung.

Der Proof-of-Concept zerlegt diesen Bereich in eigenständig deploybare Microservices. Eine Motorkonfiguration wird persistent verwaltet und anschließend durch mehrere fachlich getrennte Analyse-Services geprüft. Status und Resultate jedes Algorithmus sollen sichtbar sein. Ein einzelner fehlgeschlagener Schritt muss gezielt wiederholt werden können, ohne bereits erfolgreiche Schritte erneut auszuführen.

1.2 Architekturziele

Ziel	Konsequenz im Proof-of-Concept
Fachliche Zerlegung	Schnitt nach Business Capabilities statt nach technischen Schichten; Strategic DDD mit klaren Bounded Contexts.
Lose Kopplung	Kommunikation ausschließlich über explizite REST-Verträge; kein gemeinsam verwendetes Domain-Modul.
Ausfallsicherheit	Resilience4j Circuit Breaker kapselt nicht erreichbare Analyse-Services und begrenzt Fehlerkaskaden.
Independent Deployability	Jeder Microservice besitzt einen eigenen Build und läuft in einem eigenen Docker-Container.
Monitorability	Status, Einzelresultate und Gesamtergebnis sind über eine Analyse-ID nachvollziehbar.
Responsiveness	Der Start einer Analyse blockiert nicht bis zum Ende aller Algorithmen.
Testbarkeit	Happy Path, Fehlerfall und Retry sind automatisiert sowie über die API reproduzierbar.

1.3 Wesentliche fachliche Anforderungen

- Engine-Konfiguration erfassen, validieren, speichern und laden.
- Qualitätsanalyse für eine bestehende Konfiguration starten.
- Mindestens vier fachlich getrennte Analysealgorithmen als eigenständige Microservices verwenden.
- Status PENDING, RUNNING, READY und FAILED pro Algorithmus verwalten.
- Einzelresultate sowie ein *Overall Result* bereitstellen.
- Fehlgeschlagene Algorithmen einzeln wiederholen können.
- Ausfälle einzelner Analyse-Services kontrolliert behandeln.
- Den PoC mit Docker und Docker Compose betreiben.

2 Randbedingungen

2.1 Technische Randbedingungen

Technologie	Verwendung
Java 21 / Spring Boot	Implementierung der sechs Microservices und REST-Schnittstellen.
React / Vite / MUI	Web-Oberfläche für Konfiguration, Statusübersicht, Retry und Overall Result.
H2 / Spring Data JPA	Persistenz in Configuration Management und Analysis Management.
Resilience4j	Umsetzung des Circuit-Breaker-Patterns für Service-zu-Service-Aufrufe.
Docker / Docker Compose	Isolierte Bereitstellung und lokale Komposition aller Services.
JUnit 5 / Mockito / Postman	Automatisierte Tests sowie manuelle Demonstration der API.
PlantUML / TikZ	Architekturdiagramme und Dokumentation.

2.2 Projektgrenzen

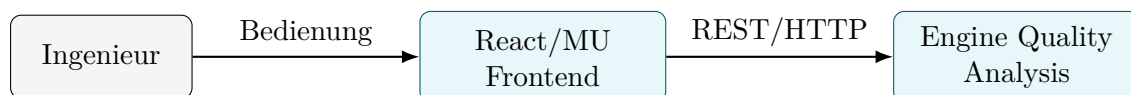
Nicht Bestandteil dieses Proof-of-Concepts sind eine reale physikalische Motorsimulation, die vollständige Migration der bestehenden Unternehmenssoftware, Kubernetes oder Cloud-Deployment sowie eine Camunda- oder MCP-Orchestration des Order-Fulfillment-Prozesses. Kafka wird im Kernumfang nicht verwendet.

Die vier Analyse-Services arbeiten daher mit vereinfachten, deterministischen Prüfregeln. Im Mittelpunkt stehen die Architekturentscheidungen, Servicegrenzen, Datenhoheit, Resilience und Nachvollziehbarkeit der Verarbeitung.

3 Kontextabgrenzung

3.1 Fachlicher Kontext

Der primäre Akteur ist der **Ingenieur**. Im Prüfungsszenario übernimmt diese Rolle der Demo-Operator. Er erstellt oder lädt eine Motorkonfiguration, startet die Qualitätsanalyse, beobachtet den Bearbeitungsstatus und stößt bei Bedarf einen Retry an.



Die Systemgrenze umfasst ausschließlich den PoC der Engine Quality Analysis. Externe Unternehmenssysteme und der vollständige Order-Fulfillment-Prozess liegen außerhalb dieser Dokumentation.

4 Lösungsstrategie

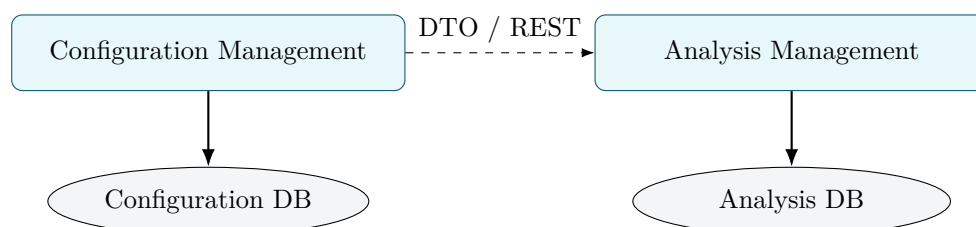
4.1 Strategic Domain-Driven Design

Die Zerlegung folgt dem Grundsatz *business capability first*. Services werden nicht nach Presentation-, Business- und Data-Layer getrennt, sondern entlang fachlicher Verantwortlichkeiten. Damit wird insbesondere das Anti-Pattern *Wrong Cut* vermieden.

Bounded Context	Microservice	Verantwortung
Configuration Management	configuration-service	Konfiguration validieren, speichern und lesen.
Analysis Management	analysis-management-service	AnalysisRun, Status, Resultate, Overall Result und Retry verwalten.
Fluid Analysis	fluid-analysis-service	Öl- und kraftstoffbezogene Konfigurationsanteile analysieren.
Thermal Analysis	thermal-analysis-service	Thermisch geprägte Konfigurationsanteile simuliert analysieren.
Electrical Analysis	electrical-analysis-service	Elektrische Konfigurationsanteile simuliert analysieren.
Engine Management Analysis	engine-management-analysis-service	Finale EMS-Analyse unter Einbezug vorheriger Resultate durchführen.

4.2 Datenhoheit und Integrationsprinzip

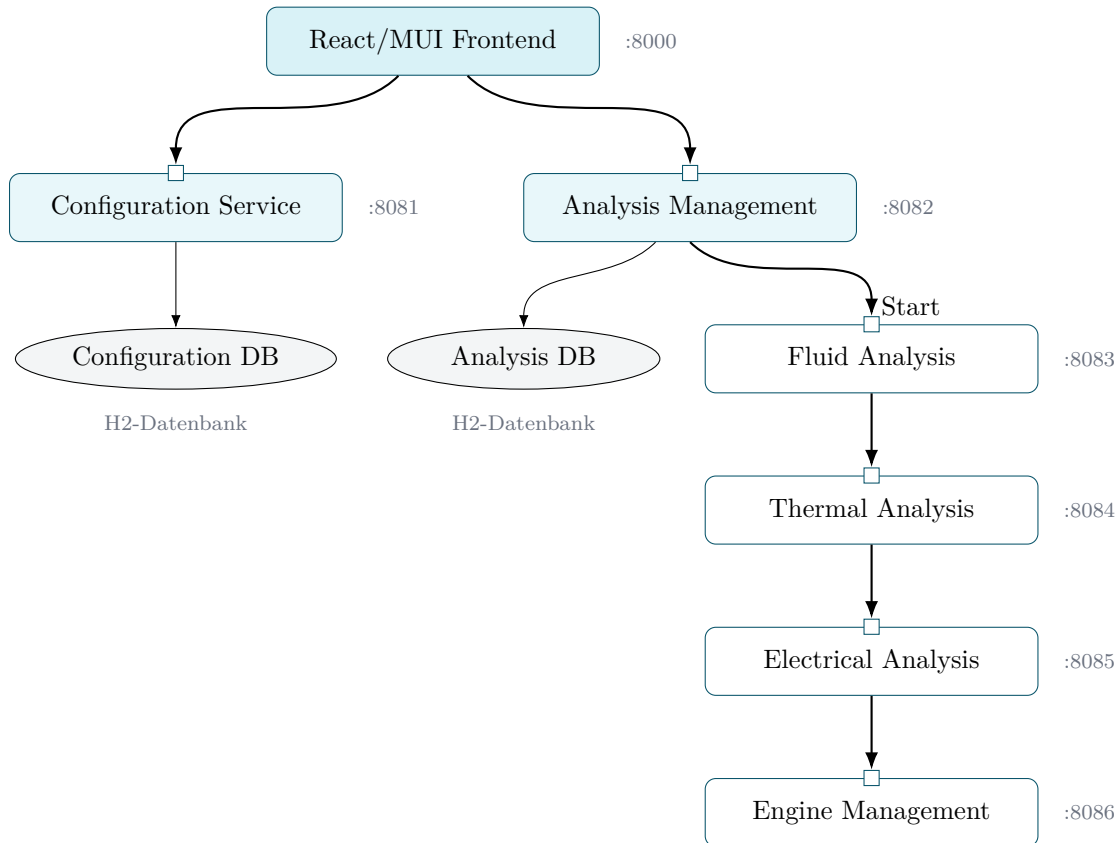
Configuration Management besitzt die Engine-Konfigurationen. Analysis Management besitzt die Analysezustände und Resultate. Die vier Analyse-Services bleiben stateless. Ein universales, gemeinsam genutztes Datenmodell wird bewusst vermieden. Integration erfolgt über DTOs und klar definierte Schnittstellen an den Context-Grenzen.



5 Bausteinsicht

5.1 Ebene 1: Systemübersicht

Die Architektur besteht aus einem Frontend und sechs Spring-Boot-Microservices. Configuration Management und Analysis Management verwalten Zustand. Die Analyse-Services sind zustandslos und bilden gemeinsam die eigentliche Qualitätsanalyse.



Alle Analyse-Services melden Status und Resultate per Callback an Analysis Management zurück.

5.2 Externe Schnittstellen

Endpoint	Bedeutung
POST/api/configurations	Neue Engine-Konfiguration anlegen.
GET/api/configurations/{id}	Gespeicherte Konfiguration laden.
POST/api/analyses	Analyse für eine Konfiguration starten.
GET/api/analyses/{id}	Status, Einzelresultate und Overall Result abrufen.
POST/api/analyses/{id}/algorithms/{algorithm}/retry	Einen fehlgeschlagenen Algorithmus gezielt wiederholen.

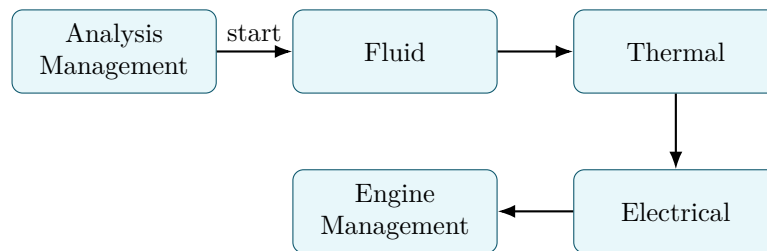
5.3 Interne Schnittstellen

Jeder Analyse-Service akzeptiert einen Analyseauftrag über `POST/internal/analyses`. Status und Resultate werden per Callback an Analysis Management gemeldet. Der Integrationsvertrag enthält die Analyse-ID, die benötigten Konfigurationsdaten und - sofern erforderlich - bereits vorhandene Ergebnisse.

6 Laufzeitsicht

6.1 Happy Path

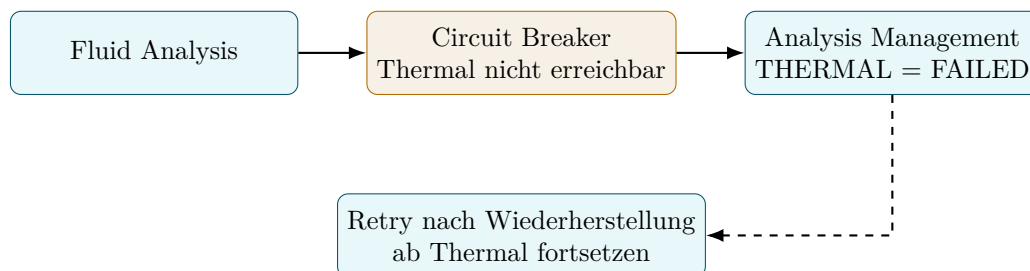
Analysis Management erzeugt den **AnalysisRun** und startet ausschließlich den Anchor-Algorithmus *Fluid*. Danach gibt jeder erfolgreiche Analyse-Service die Verarbeitung an den fachlich nächsten Service weiter. Dadurch entsteht eine sequenzielle Choreographie. Jeder Schritt meldet seinen Status zusätzlich an Analysis Management zurück.



1. Analysis Management legt einen neuen AnalysisRun mit initialen Zuständen an.
2. Fluid Analysis startet und meldet **RUNNING**.
3. Nach erfolgreichem Abschluss meldet Fluid **READY/OK** und startet Thermal.
4. Thermal und Electrical arbeiten analog weiter.
5. Engine Management bildet den letzten fachlichen Schritt und kann vorherige Ergebnisse berücksichtigen.
6. Sobald alle vier Algorithmen erfolgreich sind, wird das Overall Result auf **OK** gesetzt.

6.2 Fehlerfall, Circuit Breaker und Retry

Beim Ausfall von Thermal darf der Fehler nicht die gesamte Anwendung blockieren. Der ausgehende Aufruf wird durch Resilience4j geschützt. Der betroffene Schritt wird als fehlgeschlagen markiert und kann später gezielt wiederholt werden.



Bereits erfolgreiche Vorgänger werden beim Retry nicht erneut ausgeführt. Nach einem erfolgreichen Thermal-Retry wird die Choreographie über Electrical bis Engine Management fortgesetzt.

7 Verteilungssicht

Jeder Microservice wird als eigener Docker-Container betrieben. Die Container kommunizieren über Service-Namen im gemeinsamen Docker-Compose-Netzwerk. Dadurch müssen keine IP-Adressen im Source Code fest verdrahtet werden.

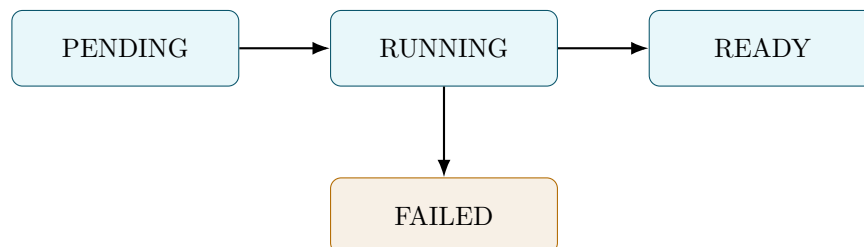
Container	Port	Zustand
configuration-service	8081	persistentes eigenes Volume
analysis-management-service	8082	persistentes eigenes Volume
fluid-analysis-service	8083	stateless
thermal-analysis-service	8084	stateless
electrical-analysis-service	8085	stateless
engine-management-analysis-service	8086	stateless

7.1 Deployment-Prinzipien

- Eigener Container pro Microservice.
- Getrennte Persistenz für Configuration Management und Analysis Management.
- Stateless Analyse-Services für einfacheres unabhängiges Starten, Stoppen und Skalieren.
- Compose-DNS statt fest verdrahteter IP-Adressen.
- Zwei Varianten im Repository: veröffentlichte Images und lokaler Build aus dem Source Code.

8 Querschnittliche Konzepte

8.1 Status- und Ergebnismodell



Resultate sind OK oder FAILED. Das Overall Result bleibt offen, solange noch Schritte ausstehen. Es wird FAILED, sobald ein relevanter Algorithmus fehlschlägt, und OK, sobald alle vier Algorithmen erfolgreich abgeschlossen sind.

8.2 Resilience

Resilience4j implementiert den Circuit Breaker für ausgehende Service-Aufrufe. Das Zustandsmodell folgt *Closed - Open - Half-Open*. Im normalen Zustand werden Aufrufe zugelassen. Nach ausreichenden Fehlern öffnet der Breaker und blockiert weitere Aufrufe. Nach einer Wartezeit werden begrenzte Testaufrufe zugelassen.

8.3 Persistenz und Datenhoheit

Nur Configuration Management und Analysis Management besitzen langfristigen Zustand. Die Analyse-Services sind bewusst stateless. Damit bleiben fachliche Datenhoheit und Bounded-Context-Grenzen klar erkennbar.

9 Architekturentscheidungen

9.1 ADR-01: Fachlicher Schnitt statt technischer Layer

Entscheidung: Die Microservices werden nach Business Capabilities getrennt.

Begründung: Fachliche Verantwortung, Datenhoheit und Deployment-Einheit bleiben zusammen. Dies reduziert das Risiko eines *Wrong Cut*.

9.2 ADR-02: REST/HTTP für Version 1

Entscheidung: Der PoC verwendet REST/HTTP für die Service-Kommunikation.

Begründung: Die Kommunikation ist leicht nachvollziehbar, demonstrierbar und testbar. Der Trade-off ist eine stärkere temporale Kopplung als bei asynchronem Messaging.

9.3 ADR-03: Choreographie statt zentraler Orchestration

Entscheidung: Analysis Management startet nur den Anchor-Schritt. Danach gibt jeder erfolgreiche Analyse-Service die Verarbeitung weiter.

Begründung: Die geforderte Zusammenarbeit der Analysealgorithmen bleibt dezentral sichtbar.

9.4 ADR-04: Circuit Breaker mit Resilience4j

Entscheidung: Service-zu-Service-Aufrufe werden mit Circuit Breakern geschützt.

Begründung: Der Ausfall eines Analyse-Services wird lokal begrenzt und als kontrollierter Zustand sichtbar.

9.5 ADR-05: Container pro Microservice

Entscheidung: Jeder Service besitzt ein eigenes Dockerfile und einen eigenen Container.

Begründung: Dies unterstützt Independent Deployability und erfüllt die technische Semesterprojektanforderung zu Docker/Docker Compose.

9.6 ADR-06: Keine Shared Persistence

Entscheidung: Configuration Management und Analysis Management besitzen getrennte Daten. Analyse-Services bleiben stateless.

Begründung: Die Kopplung zwischen Bounded Contexts wird reduziert; eine gemeinsame Datenbank wird nicht als Integrationsmechanismus verwendet.

10 Qualitätsanforderungen und Architecture Smells

10.1 Priorisierte Probleme und Gegenmaßnahmen

Prio	Smell / Anti-Pattern	Behandlung im Proof-of-Concept
1	Isolation of Failures	Circuit Breaker, kontrollierter Fehlerzustand und gezielter Retry.
2	Independent Deployability	Eigener Build und eigener Docker-Container pro Service.
3	Wrong Cut	DDD-basierte Zerlegung nach fachlichen Fähigkeiten statt nach technischen Layern.
4	Shared Persistence	Getrennte Datenhoheit und DTO-basierte Integration.

Zusätzlich adressiert die Architektur *Hard-Coded Endpoints* durch Compose-DNS und verbessert *Insufficient Monitoring* durch ein explizites Statusmodell und die UI-Sicht auf laufende Analysen. Für einen produktiven Betrieb bleiben zentralisiertes Logging, verteiltes Tracing, Authentisie-

rung/Autorisierung, API-Versionierung und ein dedizierter Observability-Stack als Ausbaustufen offen.

10.2 Qualitätsszenarien

- **Ausfall:** Thermal Analysis ist nicht erreichbar. Der Aufrufer bricht kontrolliert ab, markiert den Schritt als **FAILED** und die übrige Anwendung bleibt bedienbar.
- **Retry:** Nach Wiederherstellung wird nur der fehlgeschlagene Algorithmus erneut gestartet und die Kette anschließend fortgesetzt.
- **Nachvollziehbarkeit:** Zu jeder Analyse-ID sind Status und Resultate aller vier Algorithmen abrufbar.
- **Deployment:** Ein einzelner Analyse-Service kann unabhängig neu gebaut oder neu gestartet werden.

11 Risiken und technische Schulden

- Die REST-basierte Choreographie erzeugt temporale Kopplung zwischen aufeinanderfolgenden Services. Für eine produktive Variante ist asynchrones Messaging, z. B. Kafka, zu prüfen.
- H2 ist für den lokalen Prototyp geeignet, nicht jedoch für eine produktive Datenhaltung mit Hochverfügbarkeit.
- Monitoring ist im PoC fachlich sichtbar, aber noch kein vollständiger Observability-Stack mit zentralem Logging, Metriken und Tracing.
- Sicherheitsaspekte wie Authentisierung, Autorisierung und feinere API-Verträge sind für einen Produktivbetrieb weiter auszubauen.

12 Quellenbasis

Die Dokumentation basiert auf den Übungsblättern Nr. 4, 5 und 6, den finalen Semesterprojekt-Hinweisen, der Fallstudie *WirSchiffenDas*, Schirgi/Brenner (2021): *Quality Assurance for Microservice Architectures* sowie Garriga (2018): *Towards a Taxonomy of Microservices Architectures*. Ergänzende Projektdokumentation befindet sich in `docs/arc42.md`, `docs/ddd.md`, `docs/requirements.md`, `docs/architecture.md`, `docs/testing.md` und `docs/diagrams/` in unserem Branch auf der [GitHub-Repository](#).